

Analyse Comparative : Notre SPN face à DES et AES — Architecture, Sécurité et Perspectives

Table of Contents

Introduction	5
I. Architecture Générale et Choix Structurels	6
1. Structure interne de DES (réseau de Feistel)	6
a. Principe du réseau de Feistel	6
b. Application concrète dans DES	6
2. Structure interne d'AES (réseau SPN)	7
a. Principe du réseau SPN	7
b. Application concrète dans AES	7
3. Structure de notre SPN.....	7
4. Comparaison Feistel vs SPN.....	8
II. Les Deux Propriétés de Shannon : Confusion et Diffusion	8
1. Définition et rôle de la confusion	8
2. Définition et rôle de la diffusion	8
3. Réalisation dans AES, DES et notre SPN	9
a. DES.....	9
b. AES	9
c. Notre SPN.....	9
4. Expériences : suppression de la S-Box ou de la permutation	9
5. Limites de sécurité avec XOR et permutation seuls	10
III. Taille de Bloc : Implications Concrètes	10
1. Signification de la taille de bloc	10
2. Paradoxe des anniversaires et collisions.....	10
3. Calcul des collisions pour notre SPN (8 bits).....	11
4. Mode ECB et vulnérabilités.....	11
5. Déterminisme et sécurité sémantique	11
IV. Taille de Clé et Sécurité Brute.....	12
1. Définition et mesure de la sécurité brute	12
2. DES : 56 bits effectifs et implications	12

3.	Force brute en 1977 vs aujourd'hui	12
4.	Principe de Kerckhoffs et extrapolation	12
5.	Limites physiques de la force brute	13
6.	Respect du principe de Kerckhoffs par notre SPN.....	13
V.	Nombre de Rounds : Sécurité vs Performance.....	14
1.	Rôle des rounds et effet avalanche	14
2.	Expérimentation sur notre SPN (1 à 4 rounds)	14
3.	Comparaison avec AES et marge de sécurité	15
4.	Impact d'un doublement des rounds dans notre SPN	15
VI.	Les S-Boxes : Cœur de la Non-Linéarité.....	16
1.	Importance de la non-linéarité.....	16
2.	Conception des S-box (AES, DES, notre SPN).....	16
3.	Bijektivité et points fixes.....	17
a.	Bijektivité	17
b.	Points fixes	17
4.	Difference Distribution Table (DDT)	17
5.	Linear Approximation Table (LAT)	17
6.	Comparaison 4 bits vs 8 bits	18
VII.	Diffusion : Permutation et Propagation des Bits.....	18
1.	Critère de diffusion complète	18
2.	Fondement mathématique de MixColumns	18
3.	Couche de permutation dans notre SPN.....	18
4.	Absence de permutation au dernier round	18
VIII.	Key Schedule : Dérivation des Sous-Clés.....	19
1.	Rôle du key schedule	19
2.	Sous-clés dans DES.....	19
3.	Expansion des sous-clés dans AES	19
4.	Sous-clés par rotations et corrélation dans notre SPN.....	20
5.	Clés faibles et vulnérabilités.....	20

IX.	Résistance aux Attaques Cryptanalytiques.....	21
1.	Attaque par force brute	21
2.	Cryptanalyse différentielle	21
a.	Définition.....	21
b.	Resistance de AES, DES et notre SPN contre cette attaque	21
3.	Cryptanalyse linéaire	22
a.	Définition.....	22
b.	Resistance de AES, DES et notre SPN contre cette attaque	23
4.	Attaque par texte clair connu/choisi	24
a.	Définition.....	24
b.	Resistance de notre SPN contre cette attaque.....	24
5.	Attaques structurelles.....	24
a.	Attaque par canal auxiliaire.....	24
b.	“Meet-in-the-middle” attaque	25
X.	Limites de Notre SPN et Pistes d’Amélioration	25
1.	Taille de bloc	25
2.	Key schedule.....	25
3.	S-Box.....	25
4.	Nombre de rounds.....	26
5.	Modes d’opération et sécurité sémantique	26
	Références	26

Introduction

La cryptographie moderne s'est construite autour de chiffrements par blocs emblématiques tels que DES et AES, qui ont marqué des étapes clés dans l'évolution des standards de sécurité. Dans ce rapport, nous cherchons à situer notre réseau de permutation-substitution (SPN) dans cette trajectoire historique et technique, en le comparant aux algorithmes modernes de chiffrement afin d'analyser et de critiquer les choix de conception réalisés.

Au fil de ce travail, nous avons mené des expérimentations et des comparaisons entre **DES, AES et notre SPN**, en termes de confusion, diffusion, sécurité sémantique, modes d'opération et robustesse pratique. Cette démarche permet de mettre en évidence les forces et les limites de notre conception, tout en proposant des pistes d'amélioration pour rapprocher notre SPN des standards modernes de la cryptographie.

I. Architecture Générale et Choix Structurels

1. Structure interne de DES (réseau de Feistel)

a. Principe du réseau de Feistel

Un **réseau de Feistel** est une construction utilisée dans les algorithmes de chiffrement par bloc, nommé d'après le cryptologue d'IBM, Horst Feistel.

Ce réseau est subdivisé en **plusieurs tours**. À chaque tour, les deux blocs sont échangés, puis l'un des blocs est combiné avec la clé, passée dans une fonction ronde bien précise. Le résultat est ensuite ajouté par XOR à l'autre bloc. Pour le tour suivant, on répète les mêmes opérations sur les deux blocs obtenus, et ainsi de suite. **L'avantage principal** de ce réseau est la structure identique du chiffrement et du déchiffrement. Cela le rend plus pratique et efficace.

b. Application concrète dans DES

Le **Data Encryption Standard** est l'un des algorithmes fondés sur le schéma de Feistel. Cet algorithme de chiffrement symétrique a été publié comme FIPS 46 en janvier 1977.

L'algorithme DES transforme un bloc de 64 bits en un autre bloc de 64 bits à l'aide d'une clé effective de 56 bits. Il commence par une permutation initiale et fixe du bloc, puis le résultat est soumis à 16 itérations d'une transformation où chaque itération dépend d'une sous-clé de 48 bits. Pour la dernière ronde, une fonction inverse de la permutation initiale est appliquée afin d'obtenir le texte chiffré.

La fonction de ronde de DES se compose de :

1. **Expansion**: 32 bits $\rightarrow$ 48 bits
2. **Mélange de clé** : XOR avec la sous-clé de 48 bits
3. **Substitution** : huit S-Boxes de 6 $\rightarrow$ 4 bits
4. **Permutation**: permutation droite de 32 bits

DES est devenu le chiffrement symétrique **dominant** dans les années 1970. Cependant, sa clé de 56 bits s'est révélée trop courte et il a été progressivement brisé de plus en plus rapidement. Cette **faiblesse** a conduit

à son **abandon** et à son **remplacement** par un algorithme plus avancé et plus sûr : l'AES.

2. Structure interne d'AES (réseau SPN)

a. Principe du réseau SPN

Un **réseau de permutation-substitution** (SPN) est une architecture utilisée dans les chiffrements par bloc. Elle consiste en une série de transformations sur le bloc en clair en entrée pour produire un bloc chiffré en sortie à travers des boîtes de substitution (S-boxes) et des boîtes de permutation.

Cela renforce la **confusion** (grâce aux substitutions non linéaire) et la **diffusion** (grâce aux permutations), ce que nous définirons plus en détail dans une section ultérieure, et qui améliorent par conséquent la sécurité du chiffrement.

b. Application concrète dans AES

L'**Advanced Encryption Standard (AES)** est un chiffrement par bloc largement utilisé, basé sur un réseau de permutation-substitution.

Il opère sur des blocs fixes de 128 bits de texte clair en appliquant une série de transformations spécifiques au travers de 10, 12 ou 14 tours selon la taille de clé choisie (128, 192 ou 256 bits respectivement). Chaque tour applique successivement quatre opérations : **Substitution des bytes, Permutation des lignes, Permutation des colonnes** et **Ajout de la sous-clé de ronde**.

Contrairement aux réseaux de Feistel utilisés par DES, AES agit directement sur le bloc entièrement, ce qui améliore la rapidité de diffusion et lui a permis de devenir le standard de la cryptographie moderne.

3. Structure de notre SPN

Notre algorithme est basé sur un réseau SPN simple qui opère sur des blocs de 8 bits en utilisant une clé de 16 bits au travers de 4 tours. Chaque tour applique trois transformations : **XOR avec une sous-clé de 8 bits, double substitution sur 4 bits et permutation**. Tous les tours suivent cette structure, sauf le tour final qui remplace la permutation par un XOR final (**whitening**). Ce choix garantit la réversibilité et simplifie le déchiffrement. Ainsi, le **Key Schedule** génère 5 sous-clés de 8 bits à partir de la clé principale de 16 bits, en utilisant des rotations circulaires à gauche.

4. Comparaison Feistel vs SPN

Aspect	Feistel	SPN
Architecture	Division en deux moitiés	Traitement du block entier
Réversibilité	Même structure pour chiffrement et déchiffrement	Chaque tour doit être inversible (S-box bijective, permutation inversible)
Fonction de ronde	Appliqué sur une moitié puis l'ajout par un XOR avec l'autre	Substitutions et permutations appliquées sur le bloc
Complexité	Plus simple à concevoir	Plus contraignant (S-Box, permutation)
Sécurité	Dépend fortement du nombre de tours et de la fonction de ronde	Plus robuste et résistant face aux attaques modernes

II. Les Deux Propriétés de Shannon : Confusion et Diffusion

1. Définition et rôle de la confusion

La **confusion** signifie que chaque bit du texte chiffré doit dépendre de plusieurs parties de la clé, ce qui masque la relation entre le texte chiffré et la clé. Son **rôle principal** est d'accroître l'ambiguïté du texte chiffré, et elle est utilisée aussi bien dans les chiffrements par blocs que dans les chiffrements par flot.

2. Définition et rôle de la diffusion

La **diffusion** signifie que si l'on modifie un seul bit du texte en clair, environ la moitié des bits du texte chiffré doivent changer, et inversement. **Le but principal** de la diffusion est de masquer la relation statistique entre le texte chiffré et le texte en clair.

3. Réalisation dans AES, DES et notre SPN

a. DES

Dans DES, la confusion est principalement assurée par des **S-Boxes** fortement non linéaires, où Chaque entrée de 6 bits est transformée en une sortie de 4 bits. Cela masque la relation directe entre la clé et le texte chiffré, rendant plus difficile toute tentative d'analyse cryptographique et renforçant la sécurité globale de l'algorithme.

En revanche, la **diffusion** est assurée par les **permutations** et par **l'échange constant des moitiés**, caractéristique propre à la structure de Feistel. Ainsi, une modification d'un seul bit dans le texte en clair se propage à de nombreux bits du texte chiffré après plusieurs tours, garantissant l'effet avalanche.

b. AES

Dans AES, la confusion est obtenue grâce à l'étape **SubBytes**, qui introduit une forte non-linéarité et garantit que chaque bit du texte chiffré dépend de plusieurs bits de la clé.

La diffusion est assurée par la combinaison des étapes **ShiftRows** et **MixColumns**, qui propagent des changements à travers les lignes et les colonnes, produisant ainsi l'effet avalanche.

c. Notre SPN

Dans notre réseau de permutation-substitution (SPN), la **confusion** est obtenue grâce aux S-Boxes appliquées sur des nibbles de 4 bits. Ces substitutions non linéaires garantissent que chaque bit du texte chiffré dépend de plusieurs bits de la clé.

La **diffusion** est assurée par une couche de permutation des bits, qui réarrange leur position afin qu'une modification d'un seul bit en entrée se propage largement dans le texte chiffré après plusieurs tours.

4. Expériences : suppression de la S-Box ou de la permutation

Les expérimentations montrent que la suppression de la S-Box entraîne **une perte totale de confusion** : la relation entre la clé et le texte chiffré devient directe et exploitable. De même, l'absence de permutation **réduit fortement la diffusion**, car les bits modifiés restent localisés et ne se propagent pas dans l'ensemble du bloc.

Ces tests confirment que confusion et diffusion sont **indispensables** pour obtenir l'effet avalanche et garantir la robustesse du chiffrement.

5. Limites de sécurité avec XOR et permutation seuls

Un algorithme basé uniquement sur des opérations de XOR et de permutation ne peut pas assurer une sécurité suffisante. Le XOR est linéaire et donc vulnérable aux attaques linéaires et différentielles, tandis que la permutation seule ne crée pas de non-linéarité. Sans S-Box, l'algorithme reste prévisible et ne masque pas efficacement les relations entre texte clair et texte chiffré.

III. Taille de Bloc : Implications Concrètes

1. Signification de la taille de bloc

Le bloc en clair est transmis à l'algorithme de chiffrement qui produit un bloc de même taille mais chiffré. Le texte en clair est divisé **en blocs** d'une taille choisie, puis chaque bloc est chiffré.

Une taille de bloc plus grande augmente la résistance aux attaques, car elle réduit la probabilité de collisions entre blocs chiffrés. Par exemple, les anciens algorithmes comme DES utilisent des blocs de 64 bits, tandis que l'AES moderne adopte des blocs de 128 bits, offrant une sécurité bien supérieure face aux attaques basées sur le paradoxe des anniversaires, dont nous parlerons dans la section suivante.

2. Paradoxe des anniversaires et collisions

En théorie des probabilités, le problème des anniversaires suggère que, dans un groupe de n personnes choisies au hasard, il est probable qu'**au moins** deux partagent la même date d'anniversaire. **Le paradoxe des anniversaires** est le fait contre-intuitif que seulement 23 personnes suffisent pour que cette probabilité dépasse 50 %.

Une application concrète de ce problème en cryptographie est l'attaque dite « **birthday attack** ». Elle utilise ce modèle probabiliste pour réduire la complexité de recherche d'une collision dans un chiffrement par blocs, puisque la probabilité de collisions augmente avec le nombre de blocs traités.

Supposons que H soit le nombre total de sorties possibles. Une attaque par anniversaires permet de trouver une collision avec une probabilité de 50 % après

environ $\sqrt{H}$ essais. C'est pourquoi les tailles de bloc trop petites sont considérées comme peu sûres, et pourquoi les algorithmes modernes adoptent des blocs plus grands afin de garantir une sécurité renforcée.

3. Calcul des collisions pour notre SPN (8 bits)

D'après la partie précédente, La probabilité d'obtenir une collision devient significative quand on a environ $\sqrt{N}$ blocs, où N est le nombre total de valeurs possibles.

Notre SPN est vulnérable aux collisions très rapidement à cause de la petite taille de bloc (8 bits) puisqu'il y a $2^8 = 256$ valeurs possibles et dès qu'on chiffre environ $\sqrt{256} = 16$ blocs avec la même clé, on peut s'attendre statistiquement à observer une collision.

4. Mode ECB et vulnérabilités

Le **mode Electronic Codebook (ECB)** est l'un des modes de fonctionnement les plus simples et les plus directs pour les chiffrements par blocs : Chaque bloc du texte clair est chiffré indépendamment avec la même clé.

Malgré sa simplicité, il introduit de nombreuses vulnérabilités. En particulier, il ne fournit aucune diffusion entre les blocs, ce qui signifie qu'il échoue à masquer les motifs des données : deux blocs identiques en clair produisent deux blocs identiques en chiffré. De plus, plus la taille du bloc est petite, plus les collisions apparaissent rapidement, conformément au paradoxe des anniversaires.

5. Déterminisme et sécurité sémantique

Notre SPN est bien **déterministe** ou pour une même entrée et une même clé, il produit toujours exactement la même sortie. Même si cette propriété assure la reproductibilité de l'algorithme, elle pose un problème majeur pour la **sécurité sémantique**.

En effet, Un adversaire peut exploiter les régularités et les répétitions introduit par l'algorithme pour établir des corrélations statistiques et déduire des informations sur le contenu. C'est pourquoi les chiffrements modernes introduisent des mécanismes probabilistes afin de produire des sorties différentes même pour des entrées identiques, garantissant ainsi une véritable sécurité sémantique.

IV. Taille de Clé et Sécurité Brute

1. Définition et mesure de la sécurité brute

La **sécurité brute** d'un chiffrement désigne sa résistance face à une attaque par force brute, c'est-à-dire la tentative systématique de toutes les clés possibles jusqu'à trouver la bonne. Elle se mesure directement par la taille de l'espace de clés : pour une clé de longueur n bits, il existe 2^n combinaisons possibles, et en moyenne la moitié doit être testée avant de retrouver la clé correcte.

Plus la clé est longue, plus l'espace de recherche croît de manière exponentielle, rendant l'attaque rapidement impraticable. Ainsi, la sécurité brute est une mesure quantitative qui dépend uniquement de la taille de la clé et de la puissance de calcul disponible.

2. DES : 56 bits effectifs et implications

La clé de DES est composée de **64 bits**, dont 8 bits sont réservés à la parité et 56 bits constituent la clé effective utilisée par l'algorithme.

En 1977, l'espace de 2^{56} clés était considéré suffisant pour assurer la sécurité des systèmes de l'époque. **Cependant**, avec le temps, cette taille de la clé s'est révélée trop petite, rendant DES vulnérable aux attaques par force brute due à l'augmentation constante de la puissance de calcul.

3. Force brute en 1977 vs aujourd'hui

Époque	Puissance de calcul	Temps estimé pour casser DES
1977	Ordinateurs classiques	Jugé impraticable
1998	EFF Deep Crack	< 3 jours
Aujourd'hui	GPU / Cloud	Quelques heures/minutes

4. Principe de Kerckhoffs et extrapolation

Le **principe de Kerckhoffs** stipule qu'un système cryptographique doit rester sûr même si l'ensemble de son fonctionnement est public, à l'exception de la clé. Il s'oppose ainsi au principe de la « **sécurité par l'obscurité** », qui repose sur le secret de l'algorithme.

Aujourd'hui, la majorité des systèmes modernes suivent ce principe, car il est considéré comme plus **fiable** : la publication des algorithmes permet leur analyse par la communauté scientifique, favorise leur amélioration et renforce leur sécurité.

Tableau d'extrapolation :

Taille de clé	Essais moyens ($\sqrt{H}$)	Faisabilité
32 bits	2^{16}	Trivial
56 bits	2^{26}	Cassable
64 bits	2^{32}	Atteignable
80 bits	2^{40}	Difficile
128 bits	2^{64}	Impossible physiquement

5. Limites physiques de la force brute

Bien que les attaques par force brute puissent sembler puissantes en théorie, elles s'effondrent face aux **limites physiques du calcul**. La **limite de Landauer**, par exemple, fixe un coût énergétique minimal pour traiter ou effacer un seul bit. Même avec des superordinateurs avancés, l'énergie nécessaire pour explorer d'immenses espaces de clés dépasserait la puissance disponible à l'échelle mondiale.

Le temps constitue un autre obstacle majeur : casser une clé de 128 bits prendrait plus longtemps que l'âge de l'univers. C'est pourquoi la force brute est aujourd'hui **impraticable** et **physiquement impossible** contre les algorithmes modernes qui reposent sur des clés de 128 ou 256 bits.

6. Respect du principe de Kerckhoffs par notre SPN

Notre algorithme SPN **respecte** le principe de Kerckhoffs, car sa sécurité repose uniquement sur la clé et non sur le secret de l'algorithme. L'architecture et les mécanismes (S-Box, permutation, key Schedule) peuvent être rendus publics sans compromettre la confidentialité, puisque seule la clé doit rester secrète.

Cependant, avec une clé de 16 bits, l'espace de clés est limité à 2^{16} possibilités, ce qui reste trivial face aux ordinateurs modernes équipés d'une puissance de calcul considérable.

V. Nombre de Rounds : Sécurité vs Performance

1. Rôle des rounds et effet avalanche

L'un des fondamentaux de la cryptographie est l'utilisation de plusieurs tours (rounds), qui jouent un rôle essentiel dans la sécurité d'un système. Chaque round doit combiner la **confusion (via S-Box)** et la **diffusion (via la permutation)**. Un seul round reste trop faible car les motifs du texte clair se retrouvent encore dans le chiffré.

L'effet avalanche est une propriété recherchée et constitue un critère fondamental de sécurité dans les algorithmes de bloc : Il apparaît après plusieurs tours, lorsqu'une modification d'un seul bit en entrée entraîne le changement d'environ 50 % des bits en sortie.

2. Expérimentation sur notre SPN (1 à 4 rounds)

Pour cette partie, nous avons soumis notre algorithme à un petit test afin d'observer son comportement à chaque round et de vérifier la propriété d'avalanche.

Résultat donnée par notre SPN avec une clé 1100101010010101 :

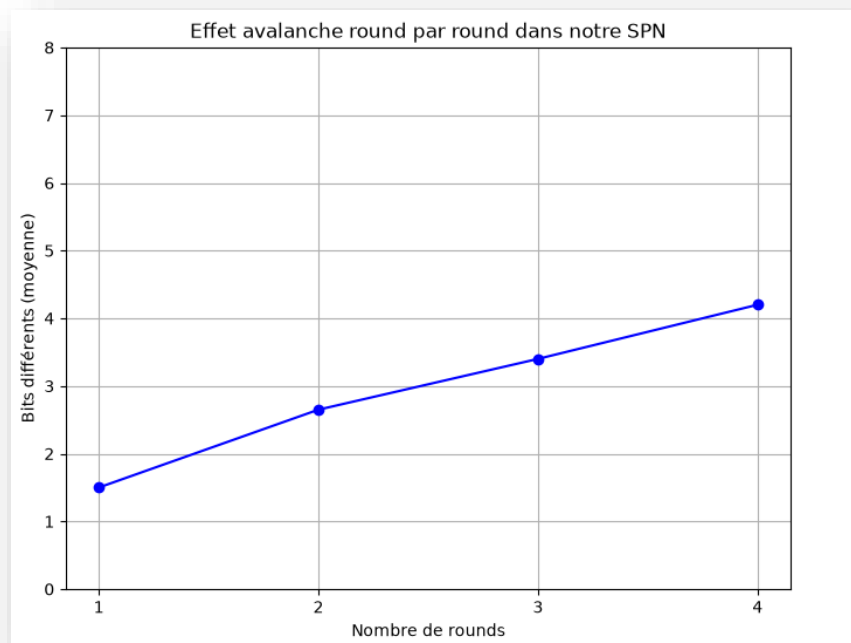


Figure 1 - graph de l'effet Avalanche tour par tour

Comme le montre le graphique, après **1 round**, la diffusion est encore trop faible (moins de 2 bits changés). Mais au fur et à mesure que le nombre de rounds augmente, la diffusion devient progressivement plus forte, jusqu'à dépasser **50 %** des bits modifiés.

Cela prouve que l'effet avalanche n'évolue pas de manière strictement linéaire, mais de façon progressive et se stabilise dans les derniers rounds. On peut ainsi remarquer que, en dessous de 4 rounds, la diffusion reste incomplète et l'effet avalanche insuffisant. Mais, Avec 4 rounds, la dépendance est forte et la sécurité statistique est assurée. Donc, pour notre SPN, **4 rounds** sont nécessaires pour que chaque bit de sortie dépende statistiquement de chaque bit d'entrée.

Ce résultat prouve que notre SPN possède une **bonne diffusion** et un **effet avalanche solide**.

3. Comparaison avec AES et marge de sécurité

AES-192 et AES-256 ont plus de rounds qu'AES-128 afin de garantir une **diffusion complète** et une **marge de sécurité suffisante** face aux attaques, car une clé plus longue nécessite davantage de tours pour éviter des faiblesses structurelles. Des attaques théoriques existent sur des versions réduites d'AES (6 ou 7 rounds), ce qui montre que le seuil minimal de sécurité se situe autour de 8–9 rounds. Les concepteurs ont donc ajouté une marge supplémentaire — 10 rounds pour AES-128, 12 pour AES-192 et 14 pour AES-256 — afin de protéger l'algorithme non seulement contre les attaques connues mais aussi contre d'éventuelles avancées futures en cryptanalyse.

4. Impact d'un doublement des rounds dans notre SPN

Une piste pour renforcer la sécurité de notre algorithme serait de doubler le nombre de rounds. À première vue, cette augmentation permettrait d'obtenir une **meilleure confusion et diffusion**, et de **stabiliser** plus rapidement l'effet avalanche.

Toutefois, cela se ferait au prix de la performance : le temps de calcul serait doublé, rendant l'algorithme plus **lent**, ce qui est peu pratique dans les systèmes modernes. De plus, même avec 8 rounds au lieu de 4, la taille de clé reste limitée à 16 bits et demeure vulnérable aux attaques par force brute.

VI. Les S-Boxes : Cœur de la Non-Linéarité

1. Importance de la non-linéarité

Les **S-Boxes** sont des composants essentiels des systèmes cryptographiques car elles apportent plusieurs propriétés fondamentales, dont la plus importante est la **non-linéarité**. Celle-ci mesure à quel point la sortie de la S-Box s'écarte d'une fonction linéaire de l'entrée. Plus cette déviation est forte, plus la S-Box résiste aux attaques par **cryptanalyse linéaire**.

Ainsi, la non-linéarité, combinée à une bonne distribution différentielle, rend les attaques par cryptanalyse différentielle beaucoup moins efficaces.

2. Conception des S-box (AES, DES, notre SPN)

Algorithme	Taille entrée/sortie	Nombre de S-Boxes	Construction	Particularités
DES	6 bits -> 4 bits	8	Tableau statique	Optimisées contre la cryptanalyse différentielle + Non linéaires mais taille réduite et vulnérable contre cryptanalyse linéaire
AES	8 bits -> 8 bits	1	Tableau dynamique (Inversion multiplicative sur GF (2 ⁸) + transformation affine)	Forte non-linéarité et bijectivité + Résistantes aux attaques linéaires et différentielles
Notre SPN	4 bits -> 4 bits (×2 en parallèle)	1	Tableau statique	Simple et facile à implémenter, mais faible taille et vulnérables aux attaques statistiques

3. Bijectivité et points fixes

a. Bijectivité

Notre S-Box est **bijective** car chaque entrée correspond à une sortie unique, et l'ensemble des sorties couvre exactement toutes les valeurs possibles sans répétition.

b. Points fixes

Un point fixe est une valeur x telle que $S(x) = x$. C'est **une faiblesse importante** dans un algorithme, car certains textes en clair peuvent produire des sorties prévisibles, ce qui réduit l'effet de confusion.

Un point fixe opposé est une valeur x telle que $S(x)$ soit **le complément binaire de x** . C'est également **une faiblesse**, car cela introduit une régularité exploitable par un attaquant : certaines entrées produisent systématiquement des sorties fortement corrélées.

4. Difference Distribution Table (DDT)

La « **Difference Distribution Table** » (**DDT**) est un outil utilisé en cryptographie symétrique pour analyser le comportement des S-Boxes. Elle recense, pour chaque différence d'entrée ΔI , la fréquence avec laquelle une différence de sortie ΔO apparaît après passage dans la S-Box. La DDT permet ainsi de mesurer la résistance d'une S-Box face à la cryptanalyse différentielle, en identifiant les biais exploités par un attaquant.

5. Linear Approximation Table (LAT)

La « **Linear Approximation Table** » (**LAT**) est un outil utilisé en cryptographie symétrique pour analyser le comportement des S-Boxes. Elle recense, pour chaque différence d'entrée Δa et Δb , combien de fois la relation linéaire suivante est vérifiée : $x \cdot \Delta a = Sbox(x) \cdot \Delta b$. La LAT permet ainsi de mesurer la résistance d'une S-Box face à la cryptanalyse Linéaire.

6. Comparaison 4 bits vs 8 bits

S-box de 4 bits	S-box de 8 bits
• Dispose de 16 entrées • Non-linéarité limitée • Vulnérable aux attaques différentielles et linéaires • Sécurité faible	• Dispose de 256 entrées • Forte non-linéarité • Meilleure diffusion • Offre une sécurité robuste et prouvée

VII. Diffusion : Permutation et Propagation des Bits

1. Critère de diffusion complète

Un chiffrement satisfait ce critère lorsqu'une modification d'un seul bit en entrée affecte, après quelques tours, **la majorité des bits en sortie** (effet avalanche).

2. Fondement mathématique de MixColumns

Dans AES, l'opération **MixColumns** repose sur l'arithmétique dans $GF(2^8)$. Chaque colonne de l'état est multipliée par une matrice fixe, ce qui transforme simultanément ses quatre octets.

Cette opération garantit qu'un changement local se propage **rapidement** à toute la colonne, assurant une **diffusion complète** en deux rounds.

3. Couche de permutation dans notre SPN

Notre SPN utilise une permutation fixe des bits pour propager les changements entre les nibbles. Cela permet qu'un bit modifié influence **progressivement** l'ensemble du bloc.

4. Absence de permutation au dernier round

Comme dans AES, notre SPN ne comporte pas de permutation finale. Cette décision évite de **compliquer** le déchiffrement et **n'affaiblit** pas la sécurité, car la diffusion est déjà assurée par les rounds précédents.

VIII. Key Schedule : Dérivation des Sous-Clés

1. Rôle du key schedule

L'utilisation d'une seule clé directement dans le chiffrement à chaque tour n'est pas pratique et a montré ses limites : elle ne garantit ni une bonne diffusion ni une bonne confusion. C'est pour cette raison que le **key schedule** a été introduit. Son rôle est de dériver, à partir d'une clé maîtresse, une série de **sous-clés différentes** utilisées à chaque round.

Ce mécanisme renforce la sécurité en évitant que la même transformation se répète à chaque étape, ce qui rendrait l'algorithme vulnérable à des attaques structurelles. Le key schedule joue donc un rôle primordial dans la résistance aux cryptanalyses, notamment **linéaires** et **différentielles**, car il assure que chaque round exploite une sous-clé distincte, réduisant les corrélations et augmentant la complexité des relations entre entrée et sortie.

2. Sous-clés dans DES

- La clé initiale est de 64 bits : 56 bits sont retenus par PC-1, les 8 autres sont ignorés ou utilisés comme bits de parité.
 - Les 56 bits sont divisés en **deux moitiés** de 28 bits et Chaque moitié subit une rotation circulaire gauche d'un ou deux bits selon le round.
 - Après rotation, PC-2 sélectionne 24 bits de chaque moitié pour finalement obtenir un sous-clé de 48 bits.
 - Ces opérations se répètent à chaque round, produisant 16 sous-clés, toutes **différentes** mais **corrélées** à la clé principale.
 - Le seul inconvénient du key schedule de DES est l'existence **de clés faibles**, qui rendent l'algorithme plus facile à attaquer.
- **PC (Permuted Choice)** : sont des "filtres" qui choisissent certains bits de la clé et les réarrangent.

3. Expansion des sous-clés dans AES

- Clé principale de 128, 192, or 256 bits.
- Nous divisons la clé principale en **mots** de 32 bits : 4 mots pour AES-128 / 6 mots pour AES-192 / 8 mots pour AES-256.
- AES génère une clé étendue composée de nombreux mots, et le nombre total de mots dépend du nombre de tours.

- Chaque mot subit deux transformations :
 - **RotWord**: effectue une rotation d'un mot de 4 octets vers la gauche d'un octet.
 - **SubWord**: applique la substitution de la S-Box AES à chaque octet du mot.
- Cette méthode garantit que chaque sous-clé est **indépendante** de la clé principale et distincte des autres sous-clés, ce qui renforce considérablement la **robustesse** et la **résistance** contre les attaques cryptanalytiques.

4. Sous-clés par rotations et corrélation dans notre SPN

- Clé principale de 16 bits.
- Génération de 5 sous-clés de 8 bits par rotations circulaires gauches.
- Avantage : **Simple** et **facile** à implémenter
- Inconvénient :
 - Les sous-clés sont **fortement corrélées** entre elles et avec la clé initiale.
 - Cette corrélation limite l'indépendance des sous-clés et rend l'algorithme plus vulnérable aux attaques ciblant le **key schedule**.

5. Clés faibles et vulnérabilités

Une **clé faible** est une clé telle que son utilisation dans une procédure de chiffrement produit un comportement indésirable. Ces clés sont généralement peu nombreuses par rapport à l'ensemble des clés possibles et même presque inexistant dans les algorithmes modernes comme AES.

En revanche, pour **DES**, l'existence de clés faibles constitue l'un des défauts majeurs de l'algorithme : certaines clés génèrent des sous-clés trop corrélées, ce qui facilite certaines attaques.

Pour **Notre SPN**, La clé principale est très, constituée seulement de **16 bits**, ce qui augmente le risque de clés faibles. Par Exemple, une clé composée uniquement par des 0 ou par des 1 peut générer des sous-clés fortement corrélées, réduisant la sécurité et facilitant la cryptanalyse.

IX. Résistance aux Attaques Cryptanalytiques

1. Attaque par force brute

Attaque par force brute est une méthode de cryptanalyse qui consiste à tester systématiquement **toutes les clés possibles** jusqu'à trouver la bonne.

- **AES**: Il est physiquement **impossible** de casser une clé AES de 128 bits par force brute, en raison de la limite de Landauer mentionnée dans les sections précédentes.
- **DES** : Avec ses 56 bits, DES est aujourd'hui considéré comme **cassable en quelques heures ou minutes** grâce à la puissance des ordinateurs modernes. Déjà en 1998, l'EFF avait réussi à casser DES en moins de 3 jours avec une machine dédiée (*Deep Crack*).
- **Notre SPN** : Avec une clé de seulement 16 bits, l'espace de recherche est si petit (65 536 clés) qu'un ordinateur classique peut casser la clé **en quelques secondes** (environ 2 secondes d'après notre programme).

2. Cryptanalyse différentielle

a. Définition

La **cryptanalyse différentielle** est une méthode générique de cryptanalyse, découverte en 1980 par Eli Biham et Adi Shamir , qui peut être appliquée aux algorithmes de blocs. Elle sert à découvrir où l'algorithme montre un comportement prédictible afin de retrouver la clé secrète.

Le principe est simple : on choisit deux textes clairs qui diffèrent par une différence ΔI , on chiffre ces deux textes et on observe la différence de sortie ΔO . En répétant l'opération sur de nombreuses paires, on **obtient une distribution statistique des différences** ; si certaines apparaissent plus souvent que prévu, cela révèle un biais exploitable pour réduire l'espace de recherche de la clé.

b. Resistance de AES, DES et notre SPN contre cette attaque

AES : Conçu après la découverte de la cryptanalyse différentielle, Ses S-boxes ont été bien développés sur une base mathématique solide afin d'assurer une diffusion uniforme et d'éliminer les biais différentiels.

DES : Lors de sa conception, les S-Boxes ont été ajustées pour limiter les biais différentiels pour rendre les attaques différentielles moins efficaces, mais n'est pas complètement résistant.

Notre SPN : D'après le graphe ci-dessous, Il existe des couples de différence entrées/sorties qui possède un nombre d'apparition élevée égale à 3 et même égale à 4, ce qui indique des biais exploitables.

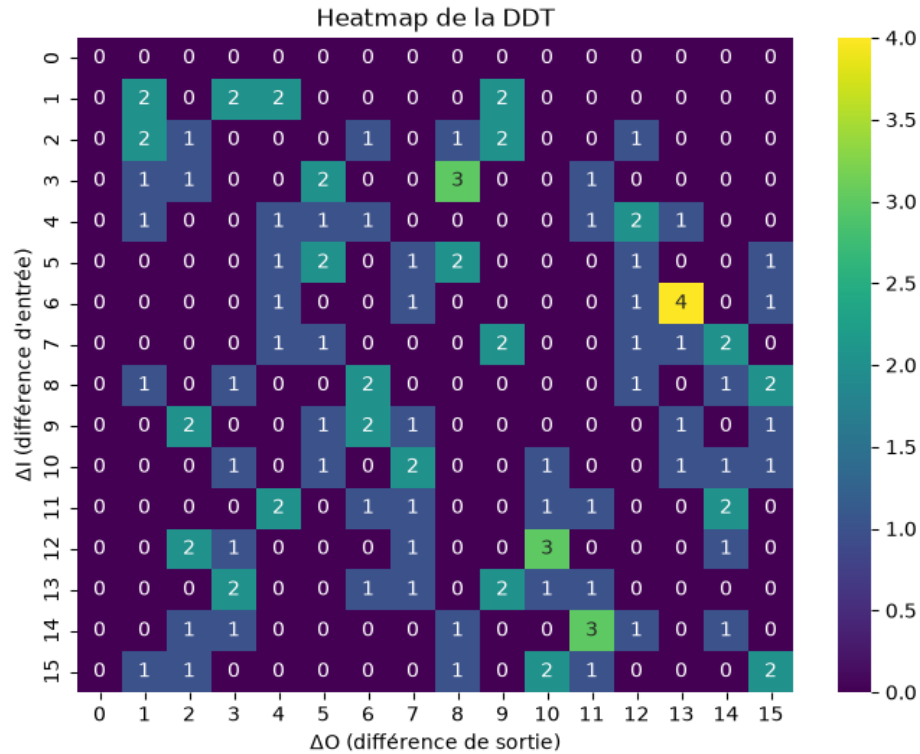


Figure 2 - Heatmap représentant DDT de notre SPN

3. Cryptanalyse linéaire

a. Définition

La **cryptanalyse linéaire** est une attaque à texte clair connu, inventée par *Mitsuru Matsui* en 1993. Elle cherche des **approximations linéaires** entre les bits du texte clair, du texte chiffré et de la clé. L'objectif est de trouver une relation qui tient avec une probabilité $p \neq 1/2$.

Le principe est simple : on cherche à approximer le comportement de la S-Box par des relations linéaires entre certains bits d'entrée et de sortie.

Concrètement, on calcule la probabilité qu'une combinaison linéaire de bits en entrée soit égale à une combinaison linéaire de bits en sortie. En répétant l'opération sur de nombreuses paires clair/chiffré, on obtient une **distribution statistique des corrélations** ; si certaines relations apparaissent avec une probabilité significativement différente de 50 %, cela révèle un biais exploitable pour réduire l'espace de recherche de la clé.

b. Resistance de AES, DES et notre SPN contre cette attaque

AES : Ses S-Boxes sont construites de manière rigoureuse pour résister aux attaques linéaires. Elles reposent sur l'inverse multiplicatif dans le corps fini $GF(2^8)$, suivi d'une transformation affine. Cette construction garantit une distribution uniforme des entrées et sorties, minimise les biais linéaires et rend les approximations statistiques inefficaces.

DES : Bien qu'il ait été conçu pour résister aux attaques différentielles, la cryptanalyse linéaire a révélé ses faiblesses. En 1993, Matsui a démontré que DES pouvait être compromis à l'aide d'environ 2^{43} textes clairs connus, une exigence qui est aujourd'hui réalisable avec la puissance de calcul moderne.

Notre SPN : D'après le graphe ci-dessous, certaines approximations linéaires présentent des biais significatifs (avec des valeurs atteignant ± 6 et même ± 8). Ces corrélations élevées indiquent que des relations linéaires entre bits d'entrée et de sortie apparaissent trop souvent, ce qui révèle des biais exploitables par une attaque linéaire.

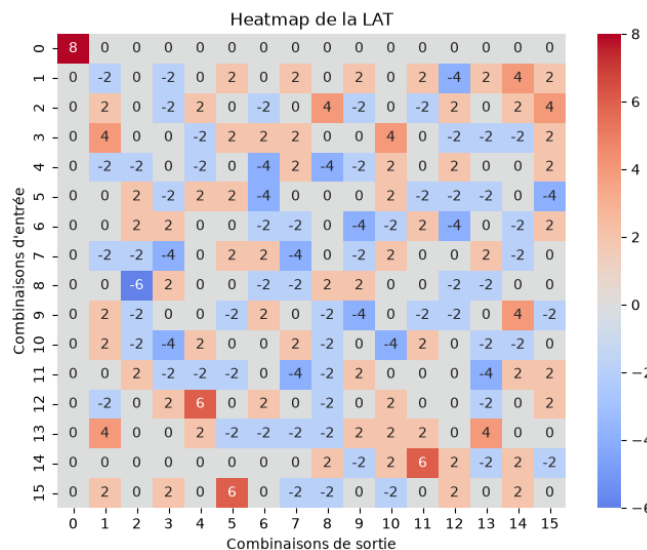


Figure 3 - Heatmap représentant LAT de notre SPN

4. Attaque par texte clair connu/choisi

a. Définition

Une **attaque à texte clair connu** est une méthode de cryptanalyse où l'attaquant dispose d'une liste de couples **texte clair / chiffré** générés avec la même clé. L'algorithme ces couples, il peut exploiter des biais statistiques de l'algorithme afin de réduire l'espace de recherche et, potentiellement, retrouver la clé secrète.

Ainsi, Une **attaque à texte clair choisi** est une méthode de cryptanalyse où l'attaquant a accès au chiffrement de l'algorithme et peut **choisir lui-même les textes clairs** à chiffrer. En observant leurs chiffrés correspondants (en utilisant la même clé), il peut exploiter les biais statistiques de l'algorithme pour réduire l'espace de recherche et faciliter la découverte de la clé.

b. Resistance de notre SPN contre cette attaque

En dehors du fait que l'espace de clé de notre SPN est trop petit (16 bits), si un attaquant dispose d'un nombre suffisant de couples clair/chiffré, il peut réduire **drastiquement** l'espace de recherche et finir par retrouver la clé. Cela rend notre SPN **vulnérable** face à ce type d'attaques.

5. Attaques structurelles

a. Attaque par canal auxiliaire

Ce type d'attaque ne repose pas sur le **fonctionnement théorique** de l'algorithme, mais sur la **manière** dont il est effectivement implémenté. En exploitant des failles d'implémentation telles que le bruit, le temps d'exécution, consommation d'énergie, signaux électromagnétiques, accès mémoire/cache, etc., un attaquant peut parvenir à déduire des **informations cruciales**, comme une clé de chiffrement ou des données sensibles.

Pour notre SPN, il est en réalité assez **vulnérable** à ce type d'attaque cryptanalytique, car nous utilisons Python comme langage de programmation. D'où, Les opérations dépendent de la structure interne du langage et du matériel, ce qui **entraîne des variations de temps d'exécution** selon les données manipulées. Par conséquent, Un attaquant pourrait mesurer ces variations pour obtenir des informations sur la clé, ce qui rend notre implémentation exposée aux attaques temporelles.

b. “Meet-in-the-middle” attaque

Le « **meet-in-the-middle** » est une attaque à texte clair connu qui peut réduire considérablement le nombre de permutations par force brute nécessaires pour déchiffrer un texte ayant été chiffré avec plus d’une clé.

Elle est particulièrement efficace contre les algorithmes à couches multiples de chiffrement, tels que le **Double-DES** et même le **Triple-DES**. L’attaquant tente de chiffrer le texte clair avec différentes clés afin d’obtenir des textes chiffrés intermédiaires, tout en déchiffrant simultanément le texte chiffré avec d’autres clés. Lorsqu’une correspondance apparaît entre les deux résultats intermédiaires, il est fortement probable que la clé utilisée pour chiffrer le texte clair et celle utilisée pour déchiffrer le texte chiffré soient les deux clés de chiffrement du bloc.

Notre SPN n’utilise qu’une seule clé et ne repose pas sur un double chiffrement. Donc, Le « meet-in-the-middle » ne **s’applique** pas, sauf si une variante multi-clé était appliquée.

X. Limites de Notre SPN et Pistes d’Amélioration

1. Taille de bloc

Limite : La taille de bloc est très petite (8 bits), ce qui entraîne des collisions rapide réduit la sécurité.

Piste d’amélioration : passer à une taille de bloc de 64 or 128 bits pour renforcer la sécurité sémantique et pour minimiser les collisions.

2. Key schedule

Limite : Le key schedule est trop simple, ce qui crée des corrélations entre les sous-clés et facilite certaines attaques.

Piste d’amélioration : Concevoir une key schedule plus robuste comme ce de AES avec les rotations, substitutions pour générer des sous-clés indépendantes et imprévisibles.

3. S-Box

Limite : La S-Box est limitée en termes de non-linéarité (4 bits) et vulnérable aux attaques différentielles et linéaires.

Piste d'amélioration : Utiliser une S-Box 8 bits construite sur $GF(2^8)$, bijective et optimisée pour réduire les biais dans la LAT et la DDT.

4. Nombre de rounds

Limite : 4 tours ne suffisent pas pour atteindre un effet avalanche complet plus rapidement et garantir une diffusion robuste.

Piste d'amélioration : Augmenter le nombre de tours selon la taille de bloc : par exemple 10 tours pour un bloc de 64 bits et 14 tours pour un bloc de 128 bits.

5. Modes d'opération et sécurité sémantique

Limite : Le chiffrement est déterministe, ce qui révèle les répétitions et compromet la sécurité sémantique.

Piste d'amélioration : Intégrer des modes d'opération modernes (CBC, CTR, GCM) avec un vecteur d'initialisation (IV) aléatoire pour garantir la sécurité sémantique et masquer les régularités.

Références

https://fr.wikipedia.org/wiki/R%C3%A9seau_de_Feistel

https://bnaskrecki.faculty.wmi.amu.edu.pl/crypto/book/part5_information_theory/ch15_des.html

https://en.wikipedia.org/wiki/Substitution%E2%80%93permutation_network

https://en.wikipedia.org/wiki/Data_Encryption_Standard

https://en.wikipedia.org/wiki/Birthday_attack

<https://fr.eitca.org/la-cyber-s%C3%A9curit%C3%A9/eitc-est-ccf-les-fondamentaux-de-la-cryptographie-classique/applications-de-chiffrement-par-blocs/modes-de-fonctionnement-pour-les-chiffrements-par-blocs/que-fait-le-mode-ecb-aux-chiffrements-par-blocs-simples/>

<https://fr-academic.com/dic.nsf/frwiki/380719>

https://fr.wikipedia.org/wiki/Cryptanalyse_diff%C3%A9rentielle

[Attaque par canal auxiliaire — Wikipédia](#)

[What is a meet-in-the-middle attack?](#)